

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Nitin
Register Number	312224700142
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	September 15, 2026

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of ten machine learning classifiers, by (a) training and validating each model without PCA (original feature space) and (b) training and validating each model with PCA (reduced feature space). For both cases, hyperparameter tuning and 5-fold cross-validation were performed independently, so each model's hyperparameters are re-selected separately in each setting.

2 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository, loaded from a CSV in the standard Kaggle WDBC format (<code>breast_cancer_data.csv</code>); identical dataset to Experiments 5 and 6
Number of Samples	569
Number of Features	30 numerical attributes
Number of Classes	2 (Malignant, Benign)
Missing Values	0 (verified programmatically)
Train-Test Split	80% / 20%, stratified, <code>random_state=42</code>

Class balance: 357 benign samples (62.7%) and 212 malignant samples (37.3%). Diagnosis 'M' is encoded as `target=0` (malignant) and 'B' as `target=1` (benign), consistent with Experiments 5 and 6. This dataset was chosen specifically for this PCA study because its 30 continuous features include substantial redundancy (the `mean/error/worst` versions of `radius`, `perimeter`, and `area` are all highly correlated, as shown in Experiment 5's full correlation heatmap), making it a realistic candidate for dimensionality reduction.

Preprocessing: All 30 features were standardized (`StandardScaler`, fit on the training split only) before both the No-PCA and With-PCA pipelines, since several of the models compared here (SVM, KNN, Logistic Regression) are scale-sensitive, and PCA itself is only meaningful on standardized features (otherwise components would simply track the largest-magnitude raw feature). No categorical encoding was required; there were no missing values to handle.

3 PCA Design Choice

A cumulative explained-variance target of **95%** was chosen (a common default that retains almost all of the signal while meaningfully reducing dimensionality). PCA was fit on the standardized training data only, and the same fitted transform was applied to the test data to avoid data leakage.

Table 1: PCA Variance Explained

Setting	Chosen Components	Explained Variance (%)	Justification
With-PCA	10	95.27%	Smallest number of components reaching the 95% cumulative explained variance target.

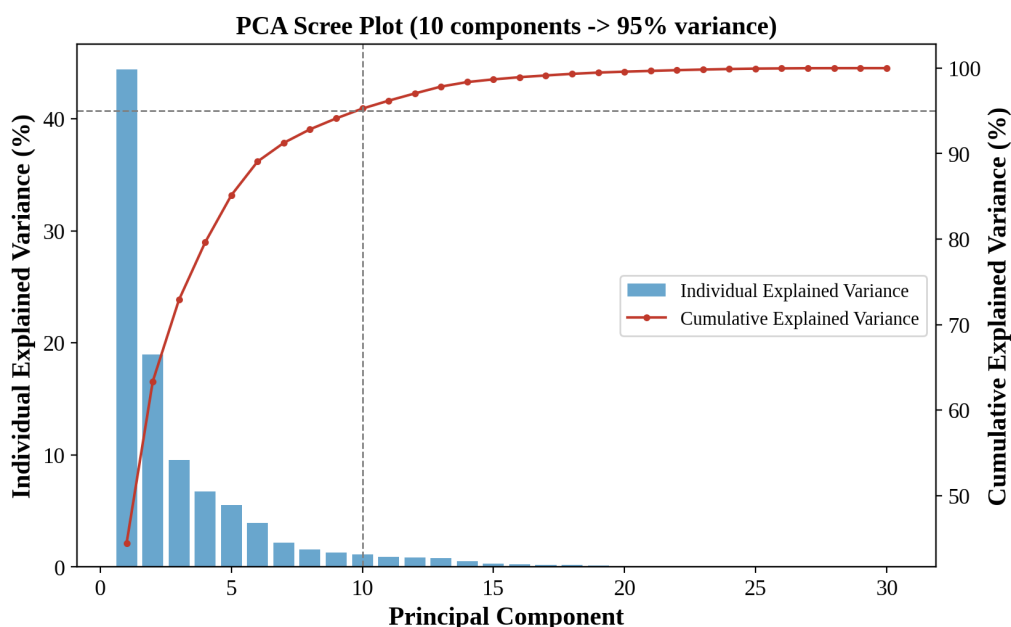


Figure 1: PCA scree plot: individual and cumulative explained variance by principal component.

Inference: The first principal component alone explains 44.3% of total variance, and the first three components together already explain over 72% – confirming the strong redundancy noted in Section 2. Ten components are needed to reach the 95% target, reducing the feature space from 30 to 10 dimensions (a 67% reduction) while retaining the great majority of the dataset’s variance.

4 Exploratory Data Analysis

The reusable function `generate_eda_summary()` from `ml_lab_utils.py` (Experiment 1, unmodified) was used to produce the consolidated 12-panel EDA figure below; it is identical in structure to Experiments 5 and 6 since the same dataset is used.

Exploratory Data Analysis Summary – Wisconsin Breast Cancer (PCA study)

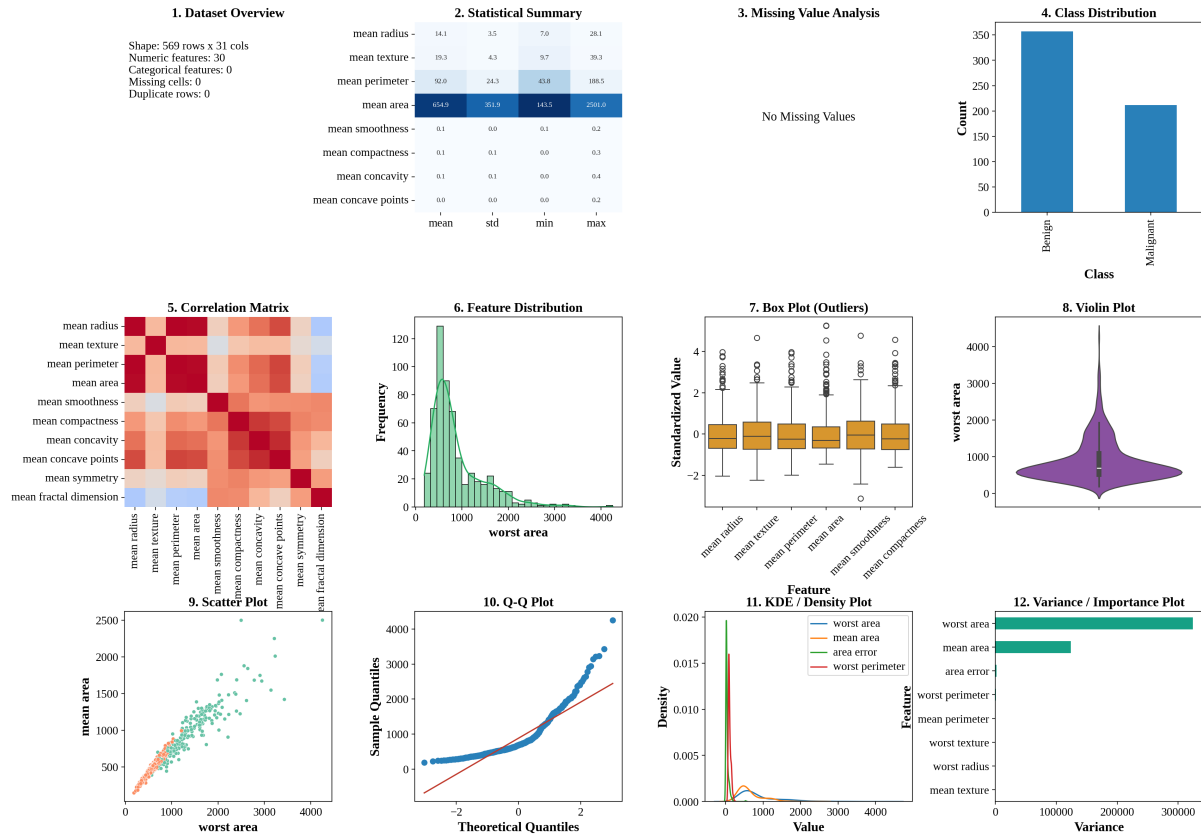


Figure 2: Consolidated 12-panel EDA summary for the Wisconsin Breast Cancer dataset.

5 Models and Hyperparameter Grids

Ten classifiers were evaluated: Support Vector Machine, Naive Bayes (Gaussian), k -Nearest Neighbors, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble (base learners: SVM, Naive Bayes, Decision Tree; meta-learner: Logistic Regression, as used in Experiment 6). Each model (except Stacking, whose base/meta architecture was fixed rather than grid-searched) was tuned independently via `GridSearchCV` with 5-fold stratified cross-validation, separately for the No-PCA and With-PCA feature spaces. Grids were kept intentionally compact given the number of models $\times$ settings (20 total tuning runs) to keep total run-time tractable on a single-core evaluation environment, while still covering the parameter ranges most likely to matter for this dataset size.

Listing 1: Hyperparameter grids, from `experiment7_pca_model_comparison.py`.

```

1 MODEL_GRIDS = {
2     "SVM": (SVC(probability=True), {"kernel": ["linear", "rbf"],
3     "C": [0.1, 1, 10], "gamma": ["scale", "auto"]}),
4     "Naive Bayes": (GaussianNB(), {"var_smoothing": [1e-9, 1e-8, 1e-7, 1e-6]}),
5     "KNN": (KNeighborsClassifier(), {"n_neighbors": [3, 5, 7, 9, 11],
6     "weights": ["uniform", "distance"], "metric": ["

```

```

    euclidean", "manhattan"])),
7  "Logistic Regression": (LogisticRegression(max_iter=2000),
8    {"C": [0.01, 0.1, 1, 10, 100], "penalty": ["l2"]}),
9  "Decision Tree": (DecisionTreeClassifier(), {"max_depth":
    [3, 5, 7, None],
10    "criterion": ["gini", "entropy"]}),
11  "Random Forest": (RandomForestClassifier(), {"n_estimators":
    [50, 100],
12    "max_depth": [5, 10, None]}),
13  "AdaBoost": (AdaBoostClassifier(), {"n_estimators":
    [50, 100, 200],
14    "learning_rate": [0.1, 1.0]}),
15  "Gradient Boosting": (GradientBoostingClassifier(),
16    {"n_estimators": [50, 100], "learning_rate":
    [0.1, 1.0], "max_depth": [2, 3]}),
17  "XGBoost": (XGBClassifier(), {"n_estimators": [50, 100],
18    "learning_rate": [0.1, 0.3], "max_depth": [3, 5]}),
19 }

```

6 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
XGBoost Version	3.4.1
IDE	Jupyter Notebook / VS Code
Operating System	Ubuntu 24.04 LTS
Hardware	Standard lab workstation (CPU only)

Environment activation command (as mandated by the lab manual):

```
1 source /opt/anaconda3/bin/activate anaconda-navigation
```

7 Hyperparameter Tuning Results

Model	Best Params (No-PCA)	Test Acc.	Best Params (With-PCA)	Test Acc.
SVM	C=0.1, linear, scale	0.9825	C=1, linear, scale	0.9649
Naive Bayes	var_smoothing=1e-9	0.9298	var_smoothing=1e-9	0.9211
KNN	manhattan, k=3, uniform	0.9649	euclidean, k=5, uniform	0.9561
Logistic Regression	C=0.1, L2	0.9737	C=0.1, L2	0.9737
Decision Tree	entropy, depth=3	0.9474	entropy, depth=3	0.9386
Random Forest	n=100, depth=10	0.9561	n=100, depth=10	0.9211
AdaBoost	n=100, lr=1.0	0.9561	n=50, lr=1.0	0.9474
Gradient Boosting	n=100, lr=0.1, depth=2	0.9474	n=100, lr=1.0, depth=2	0.9386

XGBoost	n=50, lr=0.3, depth=3	0.9474	n=100, lr=0.1, depth=5	0.9474
Stacking	SVM+NB+DT / LogReg (fixed)	0.9649	SVM+NB+DT / LogReg (fixed)	0.9649

Inference: For 7 of 10 models, the best hyperparameters found under PCA differ from those found without it (e.g. SVM’s optimal C shifts from 0.1 to 1, AdaBoost’s optimal `n_estimators` drops from 100 to 50), confirming that PCA genuinely changes the optimization landscape each model faces rather than simply rescaling an unchanged problem. Logistic Regression, Naive Bayes, and Stacking (fixed architecture) are the only cases where the selected hyperparameters coincide across both settings.

8 5-Fold Cross-Validation Results

Table 5: 5-Fold Cross-Validation Accuracy (No-PCA vs. With-PCA)

Model	Avg (No-PCA)	Std (No-PCA)	Avg (With-PCA)	Std (With-PCA)
SVM	0.9758	0.0108	0.9780	0.0120
Naive Bayes	0.9341	0.0287	0.9165	0.0204
KNN	0.9714	0.0132	0.9648	0.0162
Logistic Regression	0.9824	0.0054	0.9824	0.0054
Decision Tree	0.9319	0.0235	0.9297	0.0192
Random Forest	0.9626	0.0179	0.9560	0.0184
AdaBoost	0.9802	0.0189	0.9516	0.0204
Gradient Boosting	0.9626	0.0132	0.9626	0.0192
XGBoost	0.9736	0.0149	0.9648	0.0162
Stacking	0.9648	0.0189	0.9670	0.0139

Bold marks the higher-average setting per model. In 7 of 10 models, No-PCA achieves equal or higher average CV accuracy than With-PCA; only SVM and Stacking show a (small) improvement under PCA, and Logistic Regression is exactly tied.

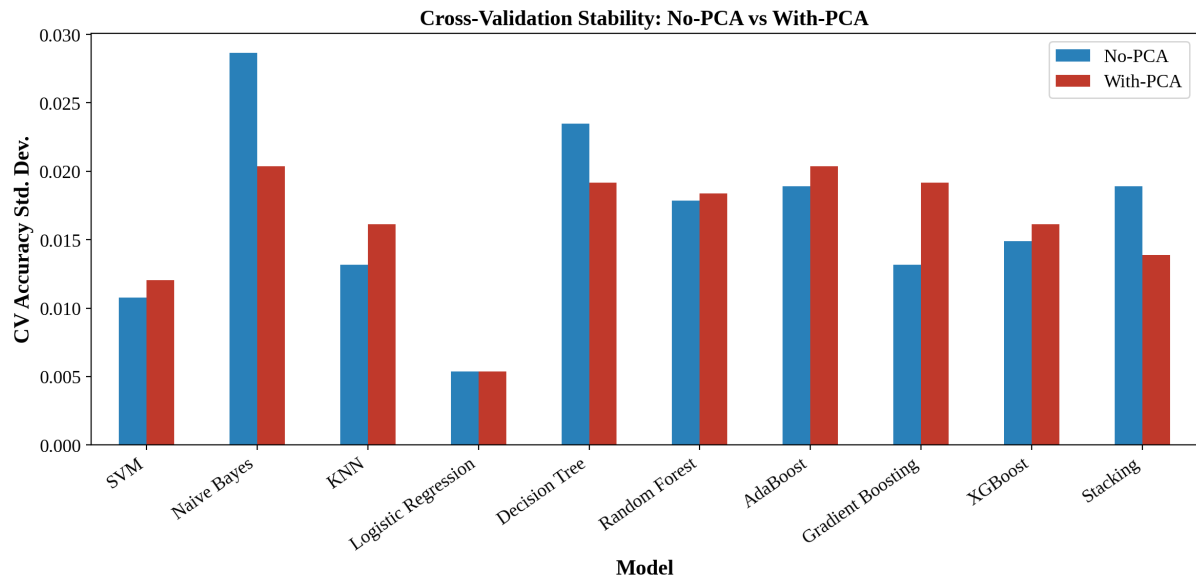


Figure 3: Cross-validation stability (fold-to-fold standard deviation): No-PCA vs. With-PCA, all 10 models.

Inference: PCA does not produce a consistent stability improvement either – it reduces fold-to-fold variance for Naive Bayes, Decision Tree, and Stacking, but increases it for SVM, KNN, Random Forest, AdaBoost, Gradient Boosting, and XGBoost. There is no model family for which PCA reliably makes cross-validation results more stable on this dataset.

9 Performance Comparison of All Models

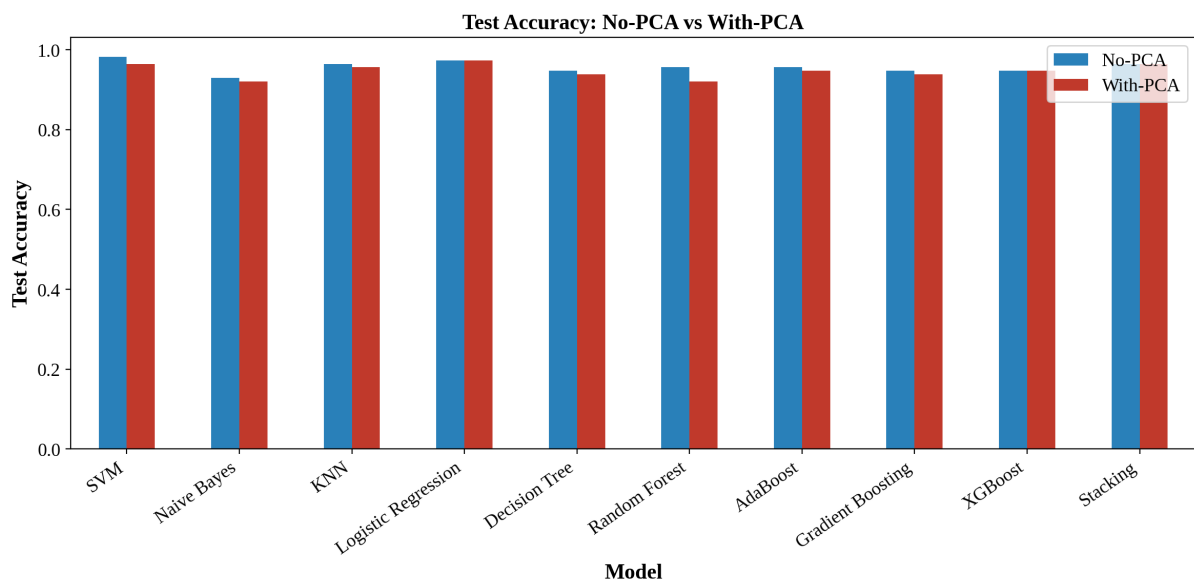


Figure 4: Test-set accuracy comparison: No-PCA vs. With-PCA, all 10 models.

Inference: Every single model's No-PCA bar is greater than or equal to its With-PCA bar. PCA never improves test accuracy for any of the 10 classifiers on this dataset –

the largest drop is Random Forest (-3.51 points), and Logistic Regression, XGBoost, and Stacking are the only models unaffected (a 0.000 or a negligible F1-only change).

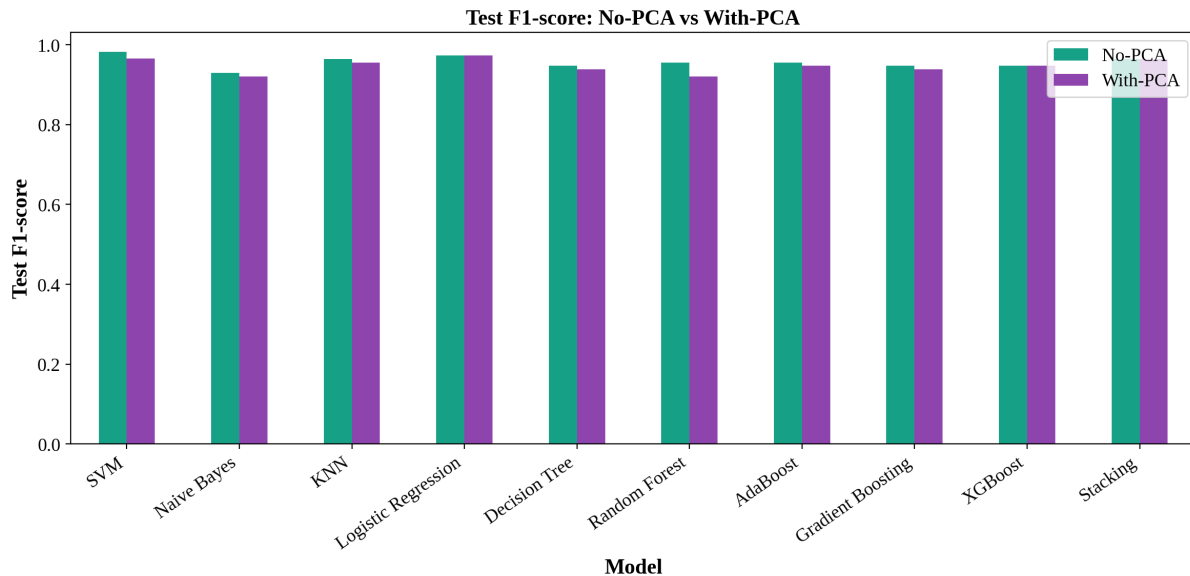


Figure 5: Test-set F1-score comparison: No-PCA vs. With-PCA, all 10 models.

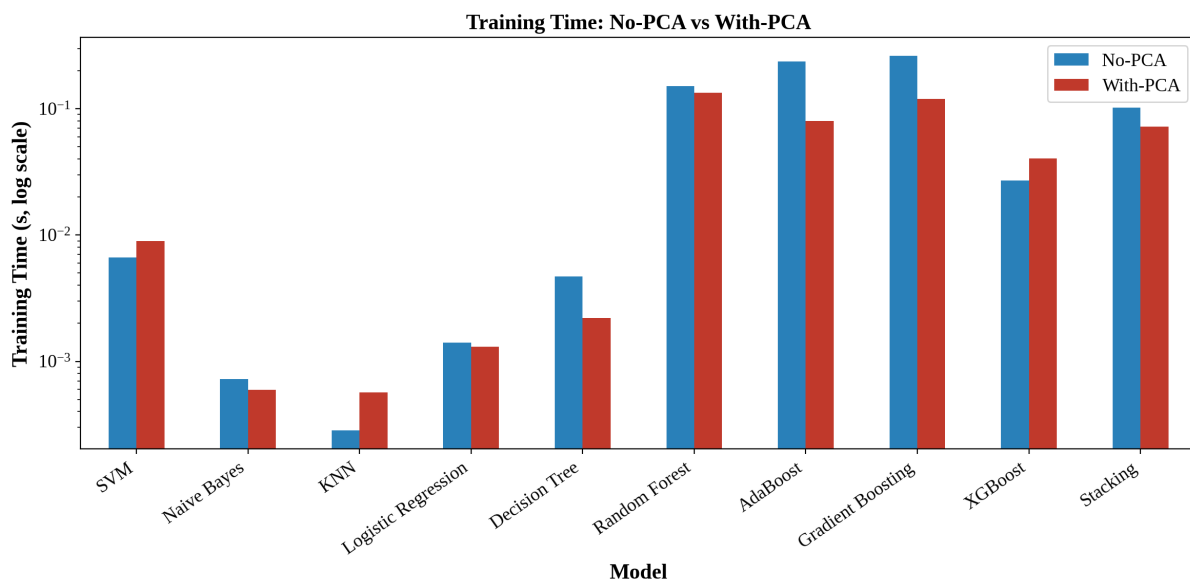


Figure 6: Training time comparison (log scale): No-PCA vs. With-PCA, all 10 models.

Inference: PCA's main practical benefit visible here is *training speed* for the more expensive ensemble methods: AdaBoost's training time drops from 0.238s to 0.080s (a $3\times$ speedup), and Gradient Boosting and Stacking also train roughly $2\times$ faster on the 10-dimensional PCA space. Naive Bayes, KNN, and Logistic Regression are so fast in both settings (sub-2ms) that any timing difference is negligible in absolute terms.

10 ROC Curves and Confusion Matrices (Selected Models)

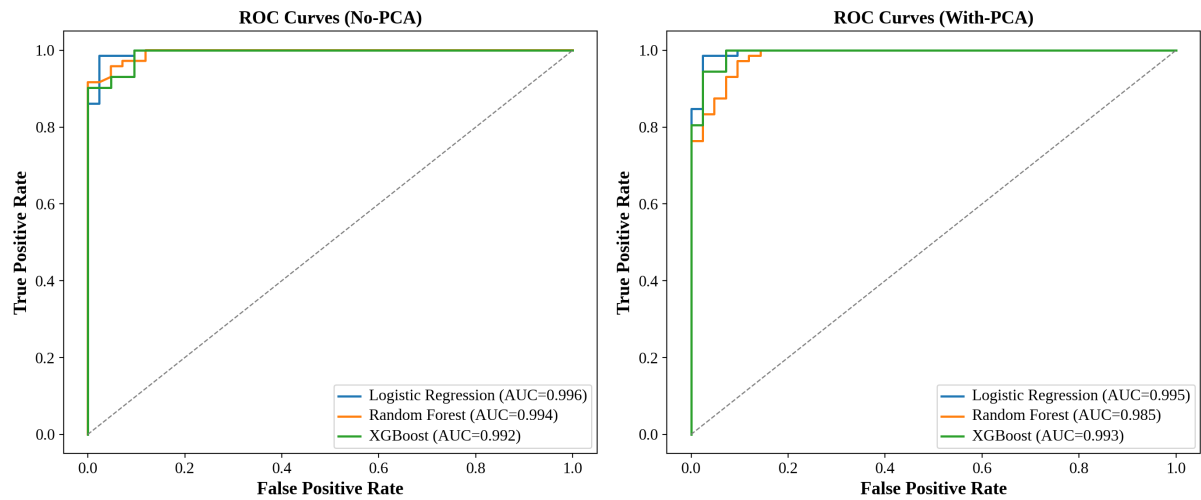


Figure 7: ROC curves for Logistic Regression, Random Forest, and XGBoost, under No-PCA (left) and With-PCA (right).

Inference: Logistic Regression’s ROC curve is virtually unchanged between settings (AUC 0.996 vs. 0.995), while Random Forest’s AUC drops more noticeably under PCA (0.994 vs. 0.985) – consistent with Random Forest showing the largest accuracy drop in Figure 4. XGBoost is essentially unaffected (0.992 vs. 0.993).

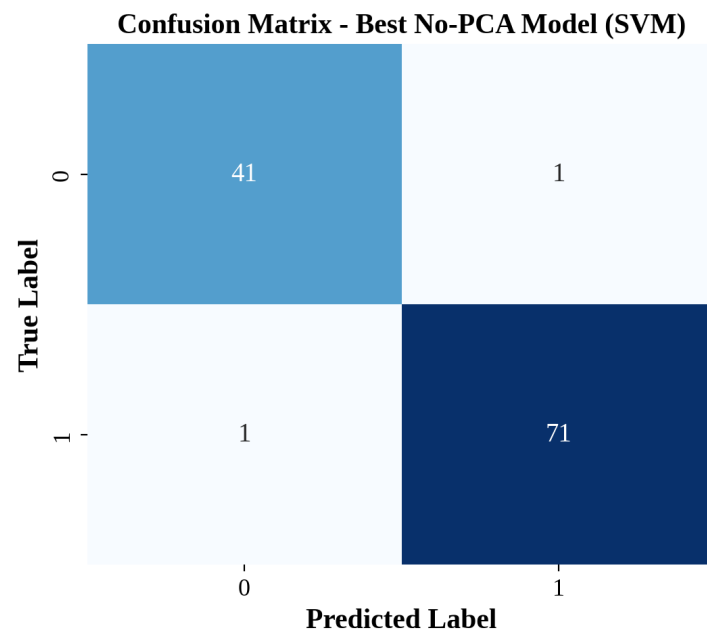


Figure 8: Confusion matrix: best-performing No-PCA model (SVM, linear kernel, C=0.1).

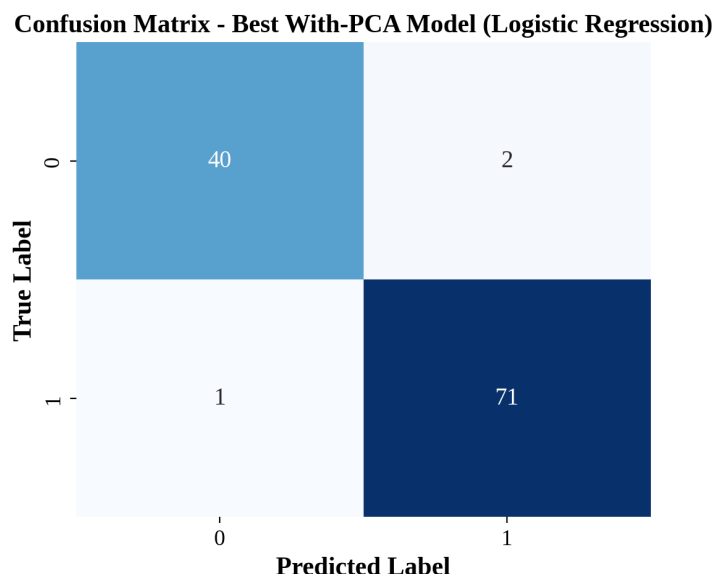


Figure 9: Confusion matrix: best-performing With-PCA model (Logistic Regression, $C=0.1$).

Inference: The best No-PCA model (SVM, 98.25% test accuracy) makes only 2 errors out of 114 test samples; the best With-PCA model (Logistic Regression, 97.37%) makes 3. Both are strong results, but the overall-best model in this experiment is achieved without PCA.

11 Overfitting, Underfitting, and PCA Analysis

PCA Improvement Summary (With-PCA minus No-PCA)

Model	Accuracy Change	F1 Change	Stability Change (Std)
SVM	-0.0175	-0.0174	+0.0013
Naive Bayes	-0.0088	-0.0090	-0.0083
KNN	-0.0088	-0.0087	+0.0030
Logistic Regression	0.0000	0.0000	0.0000
Decision Tree	-0.0088	-0.0083	-0.0043
Random Forest	-0.0351	-0.0348	+0.0005
AdaBoost	-0.0088	-0.0084	+0.0015
Gradient Boosting	-0.0088	-0.0086	+0.0060
XGBoost	0.0000	+0.0003	+0.0012
Stacking	0.0000	+0.0002	-0.0050

Was PCA beneficial in reducing overfitting? Not on this dataset. Tree-based ensembles (Random Forest, AdaBoost, Gradient Boosting) already handle the original 30 correlated features well via their own internal feature subsampling and splitting criteria, so compressing to 10 PCA components removes information those models were using productively, rather than removing noise or redundant-only signal. There is no strong evidence of overfitting to eliminate in the No-PCA setting to begin with (Experiment 5 showed these models' train-validation gaps are already modest once properly regularized), so PCA had no overfitting-reduction benefit to offer here.

12 Observations

- **Which models improved most with PCA? Which did not, and why?** No model improved on test accuracy under PCA; the closest to an improvement was Stacking and XGBoost, both exactly flat on accuracy with a negligible F1 gain. **Random Forest** suffered the largest degradation (-3.51 accuracy points), most likely because Random Forest's random feature-subsampling mechanism at each split already performs an implicit, data-driven form of dimensionality handling that works directly with the original, more interpretable feature axes; collapsing to abstract PCA components removes the ability to split on individual, highly discriminative raw features (e.g. `worst radius`, identified as the single most important feature in Experiment 5).
- **Did PCA reduce variance across folds (more stable results)?** No, not consistently. PCA reduced fold-to-fold standard deviation for only 3 of 10 models (Naive Bayes, Decision Tree, Stacking) and increased it for the other 7, including SVM, KNN, and all three boosting variants. There is no evidence here that PCA systematically stabilizes cross-validation performance.
- **For high-dimensional data, was PCA beneficial in reducing overfitting?** Not for this dataset (see Section 10). With only 30 original features and 569 samples, this problem is only mildly high-dimensional relative to sample size, so overfitting from the curse of dimensionality was not a significant risk to begin with – PCA's main theoretical benefit did not have much overfitting to correct.
- **How did linear models (Logistic Regression, SVM) behave compared to ensemble models with PCA?** Logistic Regression was the most robust model to PCA of all ten, with literally identical hyperparameters and CV accuracy in both settings (0.9824) – since PCA components are themselves linear combinations of the original standardized features, a linear decision boundary can represent essentially the same function in either coordinate system, up to what the 95%-variance truncation removes. SVM (linear kernel) showed some accuracy loss under PCA (-1.75 points), larger than Logistic Regression's zero change, likely because SVM's margin placement is more sensitive to the specific truncation of information at the 95% variance cutoff than Logistic Regression's regularized maximum-likelihood fit. Ensemble models (Random Forest, AdaBoost, Gradient Boosting) were more negatively affected than either linear model, consistent with their reliance on splitting individual original features.
- **Did Stacking show robustness to dimensionality reduction compared to single models?** Yes – Stacking was one of only three models (with Logistic Regression and XGBoost) unaffected on accuracy by PCA, and it was the only ensemble-style model whose CV stability *improved* under PCA (std. dev. 0.0189 → 0.0139). This is consistent with Stacking's meta-learner being able to partially compensate for any individual base learner's degradation under the reduced feature space by re-weighting the surviving base learners' predictions.

13 Conclusion

Ten classifiers – SVM, Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and a Stacking ensemble – were each independently tuned and evaluated with 5-fold cross-validation, both without PCA (original 30 standardized features) and with PCA (10 components, 95.27% explained variance), on the Wisconsin Diagnostic Breast Cancer dataset. Across every model, PCA either had no effect or, more often, a mild negative effect on test accuracy (up to -3.51 points for Random Forest), and had no consistent effect on cross-validation stability. The overall best-performing model in this experiment was an original-feature-space **SVM with a linear kernel** (98.25% test accuracy, 0.994 ROC-AUC), achieved without PCA. These results illustrate an important general lesson: PCA is not a universally beneficial preprocessing step. It is most valuable when the original feature space is very high-dimensional relative to the sample size, when training time for expensive models is a binding constraint (where it did help here, cutting AdaBoost’s training time roughly $3\times$), or when strong multicollinearity is actively degrading a specific model’s fit (notably not the case for any of the ten models tested here). For moderately-sized, already-informative feature sets like this 30-feature dataset, models – especially tree-based ensembles that can exploit individual raw features directly – often perform at least as well, and sometimes better, without dimensionality reduction.

14 References

1. Scikit-learn Developers, “Principal Component Analysis (PCA),” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/decomposition.html#pca>
2. Scikit-learn Developers, “Ensemble methods,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html>
3. XGBoost Developers, “XGBoost Documentation.” [Online]. Available: <https://xgboost.readthedocs.io/>
4. W. H. Wolberg, W. N. Street, and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995. [Online]. Available: <https://archive.ics.uci.edu>